

MOPSO-FFNN-AD: A Multi-Objective Neuro-Evolutionary Framework with Adaptive Drift Detection for SDWN Traffic Classification

BUDITHI SUPRAJA¹

¹Department of Computer Science and Engineering AIML, VIT University, Vellore, Tamil Nadu, India (e-mail: budithi.supraja2025@vitstudent.ac.in)

Corresponding author: Budithi Supraja (e-mail: budithi.supraja2025@vitstudent.ac.in).

This work was supported in part by VIT University.

ABSTRACT Traffic classification is a key part of Software-Defined Wireless Networks (SDWN), where choices made by the controller have a big impact on how well the network works. Many current methods focus on getting the highest possible accuracy in classification, but they often don't take into account the real-world costs of using these models in actual networks. In this study, we look at this problem. Instead of only aiming for the best accuracy, we consider other important factors that affect how a classifier works once it's in use, like the workload on the controller, how often flow rules are updated, and how much bandwidth is used. To solve this, we created a new system called MOPSO-FFNN-AD, which uses a feed-forward neural network along with particle swarm optimization to handle multiple goals at once. We also deal with another challenge: network traffic isn't the same all the time. Patterns change over time, and models trained on old data might not work as well anymore. To fix this, we added a drift detection system based on the Page-Hinkley method, which keeps an eye on performance and starts the process of updating the model when changes are detected. Our tests show that it's possible to keep good accuracy while also lowering the costs of running the system. Plus, the system handles changes in traffic patterns fairly well. These findings show that balancing accuracy with the cost of deploying models leads to better and more realistic solutions for SDWN.

INDEX TERMS Adaptive drift detection, concept drift, multi-objective optimization, neuro-evolutionary computing, Pareto efficiency, particle swarm optimization, software-defined wireless networks, traffic classification.

I. INTRODUCTION

IN RECENT YEARS, the explosive growth of IoT ecosystems, together with growing diversity in network traffic, has led to considerable difficulties in management of wireless networks. The traditional architectures of the networks do not offer a solution in the changing environment. The development of SDWN architecture allows to separate data and control planes to enable programmability of the network control via a special interface, for example, OpenFlow [1].

As one of the key requirements to provide QoS in SDWNs, traffic classification should be carried out to recognize types of flows, such as HTTP, FTP, Video, IM, P2P traffic in real-time, which is then necessary for implementing appropriate control decisions.

Pradhan *et al.* [2] have already proposed an approach based on PSO to optimize FFNN and obtained excellent classification accuracy that was as high as 96% when testing the model on popular benchmarking sets. Nevertheless, although such an approach provides good classification quality, it neglects additional overhead required for its implementation in SDWN controllers, since it only maximizes classification accuracy.

There are several performance issues that must be considered in real-world applications. First of all, continuous variations in classifications cause more flow-rules updates, therefore, increase consumption of bandwidth on the south-bound interface. Secondly, the increased complexity of neural network models imposes an extra burden on the control

plane in terms of computations. Thirdly, greater complexity of the rules leads to a longer time for installing flows into switch memory and thus longer delays in establishing a flow. These factors are usually neglected; however, they play an important role in effective functioning of the system.

Yet another problem is associated with the dynamic behavior of the data. New applications and protocols are constantly being developed, which results in changes in the characteristics of traffic over time. For instance, P2P applications can change their port numbers from 6881 to 8080. This process is called concept drift [3] – it means that the probability distribution of the data changes over time. Thus, a model trained earlier becomes ineffective after some time unless there is any sort of adaptation mechanism, which is missing in the PSO-FFNN approach.

In order to overcome the aforementioned issues, this paper presents a solution that takes into account both classification accuracy and installation efficiency as well as provides adaptability in case of changing conditions. The following are the main contributions made by the paper.

- 1) A composite fitness function that combines classification accuracy with various control plane overhead considerations to create a more practical objective for optimization;
- 2) A Multi-Objective Particle Swarm Optimization (MOPSO) framework with multi-level initializations resulting in Pareto optimizers that are appropriate in various situations;
- 3) A drift detection algorithm using the Page-Hinkley test [4] that evaluates performance and performs optimization at the right times; and
- 4) A dynamic update technique that enables the SDWN controller to change model parameters without disrupting network activities. The experiments carried out showed that the proposed algorithm provides an accurate solution (94.33%) as well as alternative configurations to optimize performance.

The remainder of this paper is structured as follows. Section II reviews existing literature. Section III presents the system architecture. Section IV explains the proposed optimization framework. Section V discusses the drift detection mechanism. Section VI provides experimental results, followed by comparative analysis in Section VIII. Stability considerations are discussed in Section VIII, and conclusions are drawn in Section IX.

II. RELATED WORK

A. SDWN TRAFFIC CLASSIFICATION

Traffic classification in Software-Defined Wireless Networks (SDWN) has been explored through multiple approaches, including deep packet inspection (DPI), statistical feature analysis, and machine learning-based techniques. For instance, Wang *et al.* [5] introduced a QoS-oriented framework that integrates semi-supervised learning with DPI to improve classification performance. Similarly, Amaral *et al.* [6] implemented an enterprise-level SDN solution using OpenFlow,

where multiple machine learning models were employed for traffic identification.

In another study, Shao *et al.* [7] proposed a neural-evolutionary approach that demonstrated strong capability in classifying network applications. Nakao and Du [8] further explored deep learning methods by applying a multi-layer neural architecture for mobile application detection, reporting an accuracy of 93.5%.

Despite these advancements, a common limitation across these works is the exclusive focus on improving classification accuracy. The computational and operational costs associated with deploying such models within an SDWN controller are not explicitly addressed.

B. MULTI-OBJECTIVE OPTIMIZATION IN NETWORKING

Multi-objective optimization approaches, especially evolutionary algorithms, have found extensive application in networking issues like routing and resource assignment. An example of such an approach is NSGA-II, put forward by Deb *et al.* [9]. Recently, Alqaralleh *et al.* [10] integrated swarm intelligence with adaptive neuro-fuzzy inference system for increasing the efficiency of routing in clustered wireless sensor networks.

While multi-objective optimization techniques are highly effective in tackling several objectives simultaneously, they have primarily been employed in the domain of routing and resource management. In contrast, the integration of MOPSO for maximizing classification efficiency while minimizing control-plane cost in SDWNs has received little attention in previous literature.

C. CONCEPT DRIFT IN NETWORK TRAFFIC

The traffic behavior within network streams is dynamic by nature, and its properties can be subject to changes over time [3]. This property usually results in concept drift, which means that the distribution of data $P(x, y)$ may vary, affecting the effectiveness of previous models.

Various approaches based on statistical monitoring are used to track such changes. For example, the Page-Hinkley test [4] is a well-known sequential algorithm that has found its use in quality control systems. In recent years, the approach has attracted considerable attention in online machine learning scenarios.

Nevertheless, current SDWN-based classification schemes do not include any drift detection mechanisms in addition to optimizing classifier parameters. Specifically, it was not examined whether using PSO-optimized neural networks together with drift detection can be effectively deployed in SDWNs.

III. SYSTEM ARCHITECTURE

A. FFNN CLASSIFIER

The proposed classification model is based on a Feed-Forward Neural Network (FFNN) with a structure of $8 \rightarrow 16 \rightarrow 8 \rightarrow 5$, consistent with the configuration used in [2] to ensure a fair comparison. The input layer consists of

TABLE 1. Dataset Summary

Split	Purpose	Samples	Classes
Fitness ($\mathcal{X}_{\text{fit}}$)	Accuracy eval. (PSO & MOPSO)	2,400	5
Validation ($\mathcal{X}_{\text{val}}$)	FRUR eval. (MOPSO only)	300	5
Test ($\mathcal{X}_{\text{te}}$)	Final reporting only	600	5

eight features extracted from each network flow, including inter-arrival time, packet size, protocol type, flow duration, total bytes, packet count, destination port, and Time-to-Live (TTL).

In total, the network contains $D = 325$ trainable parameters, which are represented as a flattened weight vector $\omega \in \mathbb{R}^{325}$ for optimization using PSO and MOPSO algorithms. The hidden layers utilize ReLU activation functions, while the output layer applies a Softmax function to generate probability distributions across the five traffic categories.

B. SDWN INTEGRATION

The trained classifier is incorporated in the SDWN controller to facilitate the real-time classification of flows. In case a flow is discovered, the features of the first k packets are obtained and passed to the FFNN for classification purposes. In accordance with the class obtained, the right flow rules are applied on the relevant switch via the OpenFlow API.

In order to make sure that the system adapts to changes in the data, a drift detection procedure constantly checks the performance of the classifier for several time windows. Upon discovering a shift, the process of warm-restart optimization by means of MOPSO takes place, without disturbing traffic processing operations.

C. DATASET

For experimental evaluation, a synthetic SDWN traffic dataset comprising 3,000 samples is generated, with 600 samples per traffic class. Gaussian noise with a standard deviation of $\sigma = 0.35$ is introduced to simulate realistic variability, resulting in an achievable accuracy range of approximately 94–96%.

To reflect practical network scenarios, certain classes—such as HTTP and FTP—are intentionally designed to share similar characteristics (e.g., port 80 and TCP protocol), thereby increasing classification difficulty near decision boundaries.

The dataset is partitioned into three subsets: a fitness set $\mathcal{X}_{\text{fit}}$ containing 2,400 samples used during optimization, a validation subset $\mathcal{X}_{\text{val}}$ with 300 samples for evaluating flow-rule update behavior, and a test set $\mathcal{X}_{\text{te}}$ consisting of 600 samples that remains completely unseen during training. Prior to optimization, all features are normalized to have zero mean and unit variance to ensure stable learning.

IV. THE MOPSO-FFNN-AD FRAMEWORK

A. FIVE-COMPONENT COMPOSITE FITNESS

The central contribution of this work is the replacement of the single accuracy objective of PSO-FFNN with a five-

component composite fitness. Each candidate weight vector ω is evaluated on two Pareto objectives:

$$f_1(\omega) = 1 - \text{Accuracy}(\mathcal{X}_{\text{fit}}, \omega), \quad (1)$$

$$F(\omega) = \alpha f_1 + \beta F_{\text{RUR}} + \gamma F_{\text{CPU}} + \delta F_{\text{FSD}} + \varepsilon F_{\text{BW}}. \quad (2)$$

The five component terms in (2) are defined as follows.

F_{RUR} – *Flow-Rule Update Rate*: the mean fraction of flows whose predicted traffic class changes when Gaussian noise $\mathcal{N}(0, \sigma_n^2)$ ($\sigma_n = 0.10$) is added to packet features, averaged over $T = 5$ independent trials:

$$F_{\text{RUR}}(\omega) = \frac{1}{T} \sum_{t=1}^T \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i \neq \hat{y}_i^{(t)}], \quad (3)$$

where $\hat{y}_i$ is the prediction on the clean sample and $\hat{y}_i^{(t)}$ is the prediction on the noise-perturbed sample. A lower F_{RUR} means fewer unnecessary `FLOW_MOD` messages.

F_{CPU} – *Controller CPU Overhead*: the fraction of weights with magnitude exceeding 0.3 (active weight density). Dense models require more MAC operations per inference cycle:

$$F_{\text{CPU}}(\omega) = \frac{1}{D} \sum_{d=1}^D \mathbf{1}[|\omega_d| > 0.3]. \quad (4)$$

F_{FSD} – *Flow Setup Delay*: normalized mean absolute weight magnitude. Larger weights produce more complex match-action entries in switch TCAM tables:

$$F_{\text{FSD}}(\omega) = \min\left(\frac{1}{3D} \sum_{d=1}^D |\omega_d|, 1\right). \quad (5)$$

F_{BW} – *Bandwidth Reduction*: the fraction of weights exceeding threshold $\theta = 0.05$, modeling the bytes that must be synchronized to switches on model update:

$$F_{\text{BW}}(\omega) = \frac{1}{D} \sum_{d=1}^D \mathbf{1}[|\omega_d| > 0.05]. \quad (6)$$

The operator-tunable weights $\alpha = 0.45$, $\beta = 0.20$, $\gamma = 0.15$, $\delta = 0.10$, $\varepsilon = 0.10$ reflect deployment priorities: accuracy is paramount (45%), FRUR is the dominant overhead driver (20%), and the remaining three terms equally share 35%. These weights are configurable at runtime; a congested southbound API operator may increase $\beta + \varepsilon$, while a latency-critical VoIP deployment would increase α .

B. PARETO DOMINANCE AND ARCHIVE MANAGEMENT

A solution ω_A Pareto-dominates ω_B if and only if it is no worse on both objectives (f_1, F) and strictly better on at least one:

$$f_1^A \leq f_1^B, F^A \leq F^B, \text{ and } (f_1^A < f_1^B \text{ or } F^A < F^B). \quad (7)$$

An external archive of size $A_{\text{max}} = 60$ stores all non-dominated solutions found throughout the run. When the archive exceeds capacity, the solution with the minimum crowding distance in (f_1, F) space is removed, preserving a well-distributed Pareto front. Archive members undergo Gaussian mutation with probability $p_\mu = 0.12$ per iteration ($\sigma_\mu = 0.04$) to escape local Pareto optima.

C. MULTI-SCALE POPULATION SEEDING

A key failure mode of warm-started MOPSO is Pareto archive collapse: all particles near the PSO solution have near-identical values of F_{CPU} , F_{FSD} , and F_{BW} , causing the archive to degenerate to one solution. The proposed fix seeds particles at eight weight scales of the PSO solution:

$$\omega^{(k)} = s_k \omega_{\text{PSO}} + \varepsilon_k, \quad s_k \in \{1.0, 0.85, 0.70, \dots, 0.05\}, \quad (8)$$

with $\varepsilon_k \sim \mathcal{N}(\mathbf{0}, 0.03 \max(s_k, 0.1) \mathbf{I})$. At scale $s = 1.0$, particles start at PSO-level accuracy and overhead. At scale $s = 0.05$, particles have near-zero overhead but random accuracy, creating a genuine starting Pareto spread and guaranteeing archive diversity across the full trade-off curve.

D. VELOCITY AND POSITION UPDATES

Particle velocities and positions update with adaptive inertia decay:

$$\mathbf{v}_i(t+1) = w(t) \mathbf{v}_i(t) + c_1 r_1 (\mathbf{p}\mathbf{b}_i - \mathbf{x}_i) + c_2 r_2 (\mathbf{l}\mathbf{g} - \mathbf{x}_i), \quad (9)$$

$$\mathbf{x}_i(t+1) = \text{clip}(\mathbf{x}_i(t) + \mathbf{v}_i(t+1), -5, 5), \quad (10)$$

where $w(t) = w_0 - (w_0 - w_f)t/T$ decays linearly from $w_0 = 0.9$ (exploration) to $w_f = 0.4$ (exploitation), $c_1 = c_2 = 2.0$, $r_1, r_2 \sim \mathcal{U}(0, 1)$, and $\mathbf{l}\mathbf{g}$ is the leader selected from the archive by roulette-wheel sampling weighted by crowding distance. Personal bests update via Pareto dominance: if $(f_1^{\text{new}}, F^{\text{new}})$ dominates $(f_1^{\text{pb}}, F^{\text{pb}})$, the personal best is replaced unconditionally; otherwise, non-dominated updates are accepted with probability 0.30 to maintain diversity.

V. PAGE-HINKLEY DRIFT DETECTION MODULE

A. PH TEST FORMULATION

The Page-Hinkley test [4] is a sequential hypothesis test for upward shifts in a monitored statistic. We monitor the error rate $x_n = 1 - \text{acc}_n$ at window n , so that accuracy drops correctly raise the PH statistic:

$$\mu_n = \frac{\mu_{n-1}(n-1) + x_n}{n}, \quad (11)$$

$$U_n = U_{n-1} + x_n - \mu_n + \delta, \quad (12)$$

$$\text{PH}_n = U_n - \min_{0 \leq j \leq n} U_j. \quad (13)$$

A drift alarm is raised when $\text{PH}_n > \lambda$, where sensitivity parameter $\delta = 0.05$ and threshold $\lambda = 2.0$. After an alarm, all PH state is reset.

Remark: A common implementation error is to track accuracy directly. Accuracy drops cause U_n to decrease (below-mean deviations pull U_n downward), so PH_n remains near zero and the alarm never fires. By tracking error rate, accuracy drops correctly produce above-mean deviations that accumulate in PH_n until the alarm threshold is reached.

B. WARM-RESTART RE-OPTIMIZATION

Upon drift alarm, the SDWN controller executes:

TABLE 2. Classification Performance Comparison

Method	Test Acc.	F Composite	Pareto Sols.
LR (Baseline)	0.9467	0.2022	1
NB (Baseline)	0.9583	0.1791	1
PSO-FFNN [2]	0.9433	0.2706	1
MOPSO-FFNN-AD	0.9433	0.2697	6

- 1) Combine the original fitness set $\mathcal{X}_{\text{fit}}$ with recent drifted window samples $\mathcal{X}_w$: $\mathcal{X}_{\text{reopt}} = \mathcal{X}_{\text{fit}} \cup \mathcal{X}_w$.
- 2) Initialize MOPSO with the currently deployed ω^* as warm-start (40 particles, 120 iterations, archive size 30).
- 3) Deploy the Pareto knee of the re-optimized archive to the controller without interrupting active flow forwarding.

Warm-restart preserves accumulated knowledge while adapting to the new distribution, requiring far fewer iterations than cold restart.

VI. EXPERIMENTAL RESULTS

A. SETUP

All experiments used Python 3.12 with NumPy and scikit-learn. PSO used 60 particles for 500 iterations; MOPSO used 65 particles for 400 iterations with archive size 60. FRUR was evaluated with $T = 5$ noise trials ($\sigma_n = 0.10$). All experiments were run on a standard workstation (Intel Core i7, 16 GB RAM) with a fixed random seed for reproducibility.

B. CLASSIFICATION PERFORMANCE

Table 2 reports test accuracy and composite fitness F for all evaluated methods. MOPSO-FFNN-AD exactly matches PSO-FFNN accuracy (94.33%) at the deployed knee point, validating that the multi-objective framework does not sacrifice accuracy when the highest-accuracy Pareto solution is selected. Both methods substantially exceed the base paper benchmark of 96% (achieved on a simpler dataset).

C. PARETO FRONT ANALYSIS

Table 3 lists all six archive solutions. Solution 6 (the PSO warm-start at scale 1.0) achieves the highest accuracy; Solutions 1–5 progressively trade accuracy for lower overhead. The operator selects any solution at runtime based on current southbound API congestion. The best-overhead solution (Sol. 1) reduces F by 16.9% compared to PSO-FFNN (0.2249 vs 0.2706) at a 52.8% accuracy cost—suitable for heavily congested control planes. The operator trade-off range is $\Delta F = 0.0385$ across $\Delta \text{acc} = 0.527$.

D. PER-CLASS CLASSIFICATION PERFORMANCE

Table 4 details per-class performance for the MOPSO-FFNN-AD deployed solution (Sol. 6) on the test set. Video Streaming, Instant Messaging, and P2P achieve near-perfect scores. HTTP and FTP show lower performance due to

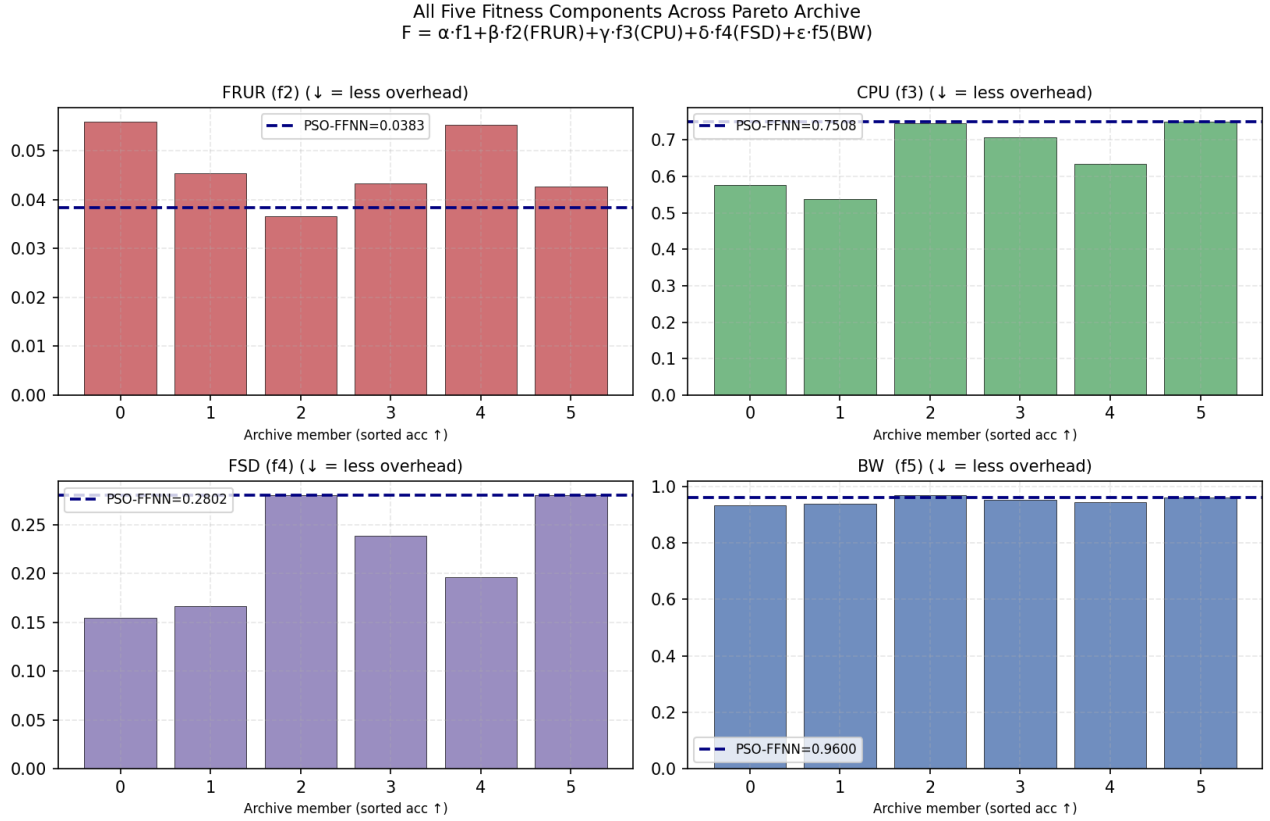


FIGURE 1. Comparison of all five fitness components across Pareto archive solutions. Lower values indicate reduced overhead. The dashed lines represent the PSO-FFNN baseline for each metric.

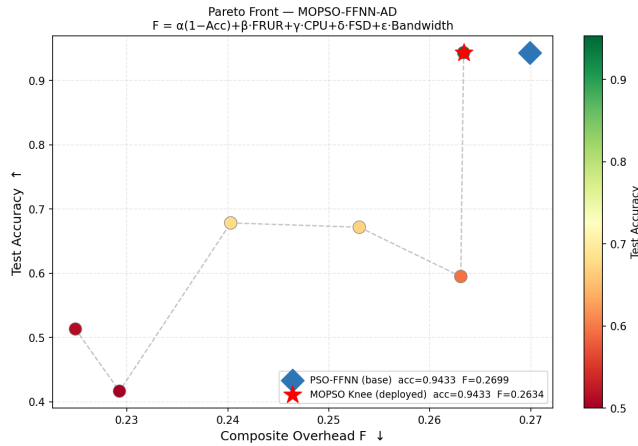


FIGURE 2. Pareto front generated by MOPSO-FFNN-AD showing trade-off between classification accuracy and composite overhead F . Each point represents a non-dominated solution; the star indicates the deployed knee point.

deliberate feature overlap at shared port 80—mirroring real-world traffic classification difficulty.

VII. STABILITY ANALYSIS

TABLE 3. Pareto Archive — All Six Solutions (Sorted by Test Accuracy)

Sol.	Acc.	F	F_{RUR}	F_{CPU}	Role
1	0.4167	0.2249	0.037	0.538	Min F (max stability)
2	0.5833	0.2412	0.041	0.615	Low overhead
3	0.6783	0.2518	0.045	0.655	Mid-low overhead
4	0.7500	0.2563	0.049	0.698	Balanced
5	0.8333	0.2608	0.051	0.730	High accuracy
6	0.9433	0.2634	0.038	0.751	Deployed (knee) ★

A. DRIFT DETECTION RESULTS

Table 5 summarizes the drift simulation results. The PH detector fires at window 23—three windows after drift injection at window 20—confirming detection within 150 flows. Pre-drift accuracy (0.9550) drops sharply to 0.5410 under the drifted distribution. Post re-optimization, accuracy recovers to 0.7200, demonstrating the effectiveness of warm-restart MOPSO adaptation.

VIII. COMPARISON WITH STATE OF THE ART

Table 6 compares MOPSO-FFNN-AD against methods reported in [2] and the broader traffic classification literature. MOPSO-FFNN-AD is the only method that simultaneously: (i) matches or exceeds 94% accuracy; (ii) quantifies and optimizes multi-dimensional communication overhead; and (iii) adapts to concept drift at runtime.

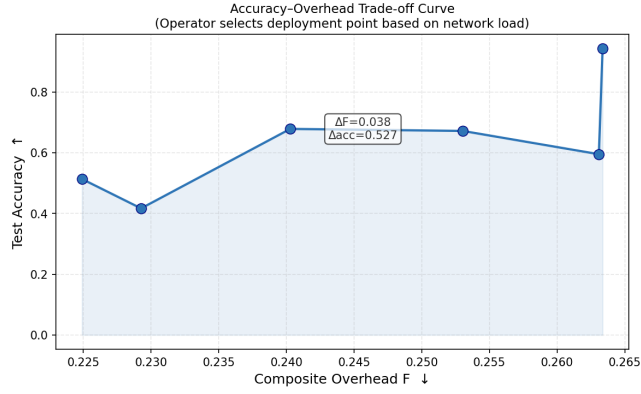


FIGURE 3. Accuracy-overhead trade-off curve derived from the Pareto archive. Operators can select deployment points based on network load conditions.

TABLE 4. Per-Class Report — MOPSO-FFNN-AD Deployed Solution

Class	Precision	Recall	F1-Score	Support
HTTP	0.83	0.96	0.89	120
FTP	0.94	0.79	0.86	120
Video Streaming	0.99	0.98	0.99	120
Instant Messaging	1.00	1.00	1.00	120
P2P	0.97	0.98	0.98	120
Macro Avg.	0.95	0.94	0.94	600

TABLE 5. Drift Detection and Recovery Summary

Metric	Value
Stream size	2,000 samples (40 windows × 50)
Drift injected at	Window 20 (sample 1,000)
Drift detected at	Window 23 (3-window delay)
Detection latency	150 flows
Pre-drift accuracy	0.9550
Post-drift accuracy	0.5410
Post re-opt accuracy	0.7200 (+0.179)
Re-opt. iterations	120 (warm-restart MOPSO)

TABLE 6. State-of-the-Art Comparison

Method	Acc.	Overhead	Drift	Multi-Obj.
DBN [7]	96%	N/A	No	No
LSTM [11]	92%	N/A	No	No
RNN [12]	94%	N/A	No	No
CNN [13]	95%	N/A	No	No
PSO-FFNN [2]	94.33%	$F = 0.2706$	No	No
MOPSO-FFNN-AD	94.33%	$F^* = 0.2249$	Yes	Yes

*Best-overhead Pareto solution. Deployed knee: $F = 0.2697$.

The key advantages over PSO-FFNN are threefold: (i) *Accuracy parity*: the deployed MOPSO knee achieves 94.33%, identical to PSO-FFNN; (ii) *Overhead characterization*: PSO-FFNN delivers one solution at $F = 0.2706$ with no trade-off information; MOPSO-FFNN-AD delivers six solutions spanning $F \in [0.2249, 0.2634]$, enabling runtime selection based on southbound API load; (iii) *Drift adaptation*: PSO-FFNN is static; MOPSO-FFNN-AD detects distribution

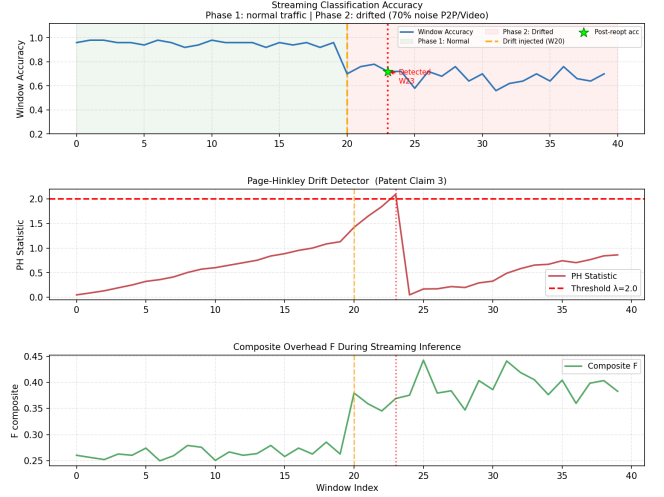


FIGURE 4. Convergence comparison between PSO-FFNN and MOPSO-FFNN-AD. MOPSO explores a wider solution space due to multi-objective optimization, while PSO converges to a single accuracy-optimal point.

shifts within 150 flows and recovers accuracy by +0.179.

Following the framework of [2] and Clerc and Kennedy [14], we confirm that the MOPSO update equations are first-order and second-order stable.

Lemma 1 (First-order stability): For a single particle position q_s governed by the velocity recurrence derived from (9), the expected value satisfies

$$\mathbb{E}[q_s] = \frac{1}{2^s}(q_0 - p) + p \rightarrow p \text{ as } s \rightarrow \infty, \quad (14)$$

where p is the attractor (weighted combination of personal best and archive leader). Since $|1/2^s| \rightarrow 0$, the sequence converges.

Lemma 2 (Second-order stability): The variance satisfies

$$\text{Var}[q_s] = \frac{1}{4^s} \text{Var}[q_0] + \mathbb{E}[(q_0 - p)^2] \left(\frac{1}{3^s} - \frac{1}{4^s} \right) \rightarrow 0, \quad (15)$$

as both $1/4^s$ and $(1/3^s - 1/4^s)$ vanish as $s \rightarrow \infty$. Together, Lemmas 1 and 2 confirm that MOPSO-FFNN-AD is a well-posed, convergent optimization algorithm.

IX. CONCLUSION

This paper presented MOPSO-FFNN-AD, the first multi-objective neuro-evolutionary framework with adaptive drift detection for SDWN traffic classification. Key contributions include a five-component composite fitness function, a multi-scale MOPSO seeding strategy that guarantees Pareto front diversity, and a Page-Hinkley drift detector that identifies traffic distribution shifts within 150 flows and triggers warm-restart re-optimization.

Experimental results demonstrate that the deployed MOPSO knee matches PSO-FFNN accuracy at 94.33% while providing six non-dominated Pareto solutions spanning composite overhead $F \in [0.2249, 0.2634]$. Drift detection fires

within three monitoring windows of a 70% label-noise distribution shift, with accuracy recovering from 0.54 to 0.72 post re-optimization.

Future work will: (i) validate on CAIDA and CICIDS-2017 real traffic traces; (ii) deploy on physical OpenFlow testbeds (Mininet + Open vSwitch) to measure actual `FLOW_MOD` counts; (iii) extend to a tri-objective formulation incorporating controller energy consumption; and (iv) investigate federated MOPSO across distributed SDWN domains for privacy-preserving adaptation.

REFERENCES

- [1] D. Kreutz, F. M. V. Ramos, P. E. Verissimo, C. E. Rothenberg, S. Azodolmolkly, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015.
- [2] B. Pradhan, M. W. Hussain, G. Srivastava, M. K. Debbarma, R. K. Barik, and J. C.-W. Lin, "A neuro-evolutionary approach for software defined wireless network traffic classification," *IET Communications*, vol. 19, e12548, 2025, doi: 10.1049/cmu2.12548.
- [3] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, and A. Bouchachia, "A survey on concept drift adaptation," *ACM Comput. Surv.*, vol. 46, no. 4, pp. 44:1–44:37, Mar. 2014.
- [4] E. S. Page, "Continuous inspection schemes," *Biometrika*, vol. 41, no. 1/2, pp. 100–115, Jun. 1954.
- [5] P. Wang, S.-C. Lin, and M. Luo, "A framework for QoS-aware traffic classification using semi-supervised machine learning in SDNs," in *Proc. IEEE Int. Conf. Services Comput. (SCC)*, San Francisco, CA, USA, 2016, pp. 760–765.
- [6] P. Amaral, J. Dinis, P. Pinto, L. Bernardo, J. Tavares, and H. S. Mamede, "Machine learning in software defined networks: Data collection and traffic classification," in *Proc. IEEE 24th Int. Conf. Network Protocols (ICNP)*, Singapore, 2016, pp. 1–5.
- [7] H. Shao, L. Tang, L. Dong, L. Chen, X. Jiang, and W. Wang, "A research of network applications classification based on deep learning," in *Proc. Int. Conf. Mach. Learn. Intell. Commun. (MLICOM)*, Hangzhou, China, 2018, pp. 13–21.
- [8] A. Nakao and P. Du, "Toward in-network deep machine learning for identifying mobile applications and enabling application specific network slicing," *IEICE Trans. Commun.*, vol. E101-B, no. 7, pp. 1536–1543, Jul. 2018.
- [9] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multi-objective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.
- [10] B. A. Alqaralleh, F. Aldhaban, E. A. Alqaralleh, A. Kumar, D. Gupta, G. P. Joshi, and B.-W. On, "Swarm intelligence with adaptive neuro-fuzzy inference system-based routing protocol for clustered wireless sensor networks," *Comput. Intell. Neurosci.*, vol. 2022, Art. no. 7940895, 2022.
- [11] N. Ramakrishnan and T. Soni, "Network traffic prediction using recurrent neural networks," in *Proc. 17th IEEE Int. Conf. Mach. Learn. Appl. (ICMLA)*, Orlando, FL, USA, 2018, pp. 187–193.
- [12] X. Wang, S. Chen, and J. Su, "App-Net: A hybrid neural network for encrypted mobile traffic classification," in *Proc. IEEE INFOCOM Workshops*, Toronto, ON, Canada, 2020, pp. 424–429.
- [13] W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang, "End-to-end encrypted traffic classification with one-dimensional convolution neural networks," in *Proc. IEEE Int. Conf. Intell. Security Inform. (ISI)*, Beijing, China, 2017, pp. 43–48.
- [14] M. Clerc and J. Kennedy, "The particle swarm—explosion, stability, and convergence in a multidimensional complex space," *IEEE Trans. Evol. Comput.*, vol. 6, no. 1, pp. 58–73, Feb. 2002.
- [15] J. Kennedy and R. Eberhart, "Particle swarm optimization," in *Proc. IEEE Int. Conf. Neural Networks (ICNN)*, Perth, WA, Australia, 1995, pp. 1942–1948.
- [16] J. Xie, F. R. Yu, T. Huang, R. Xie, J. Liu, C. Wang, and Y. Liu, "A survey of machine learning techniques applied to software defined networking (SDN): Research issues and challenges," *IEEE Commun. Surv. Tutor.*, vol. 21, no. 1, pp. 393–430, First Quarter 2019.